Alfido Tech DevOps Internship

Task 1 — Containerize a Python Web Application

Intern: KVSajith34 | Date: 17 May 2026

1. Task Overview

The objective of this task is to containerize a Python-based Flask web application using Docker. The setup includes a multi-stage Dockerfile for optimized image builds and a docker-compose.yml to orchestrate the Flask app alongside a PostgreSQL database.

Tech Stack

Component	Technology
Language	Python 3.11
Web Framework	Flask 3.0
Database	PostgreSQL 15 (Alpine)
WSGI Server	Gunicorn 21.2
Containerization	Docker 29.4.3
Orchestration	Docker Compose
OS Environment	WSL2 Ubuntu on Windows 11

2. Project Structure

```
task1-docker-webapp/
■■■ app/
■   ■■■ app.py           # Flask application
■   ■■■ requirements.txt  # Python dependencies
■■■ Dockerfile          # Multi-stage Docker build
■■■ docker-compose.yml  # App + DB orchestration
■■■ .dockerignore       # Docker build exclusions
■■■ README.md           # Documentation
```

3. Configuration Files

3.1 Dockerfile (Multi-Stage Build)

```
# Stage 1 — Builder
FROM python:3.11-slim AS builder
WORKDIR /app
COPY app/requirements.txt .
RUN pip install --upgrade pip && pip install --no-cache-dir -r requirements.txt

# Stage 2 — Runner
FROM python:3.11-slim AS runner
WORKDIR /app
```

```
COPY --from=builder /usr/local/lib/python3.11/site-packages
/usr/local/lib/python3.11/site-packages
COPY --from=builder /usr/local/bin /usr/local/bin
COPY app/ .
EXPOSE 5000
CMD ["gunicorn", "--bind", "0.0.0.0:5000", "app:app"]
```

3.2 docker-compose.yml

```
services:
  app:
    build: .
    container_name: flask-app
    ports:
      - "5000:5000"
    environment:
      - DB_HOST=db
      - DB_NAME=alfidodb
      - DB_USER=alfido
      - DB_PASSWORD=alfido123
    depends_on: [db]
    restart: always

  db:
    image: postgres:15-alpine
    container_name: postgres-db
    environment:
      - POSTGRES_DB=alfidodb
      - POSTGRES_USER=alfido
      - POSTGRES_PASSWORD=alfido123
    volumes:
      - pgdata:/var/lib/postgresql/data
    ports:
      - "5432:5432"

volumes:
  pgdata:
```

3.3 Flask Application (app.py)

```
from flask import Flask, jsonify
import psycopg2, os

app = Flask(__name__)

@app.route("/")
def home():
    return "<h1>Alfido Tech DevOps Internship</h1><p>Flask app is running in
Docker!</p>"

@app.route("/health")
```

```
def health():
    return jsonify({"status": "healthy", "service": "flask-app"})

@app.route("/db")
def db_check():
    try:
        conn = psycopg2.connect(host=os.getenv("DB_HOST", "db"), ...)
        return jsonify({"database": "connected"})
    except Exception as e:
        return jsonify({"database": "error", "detail": str(e)}), 500
```

4. Build & Run Commands

Command	Description
`docker compose up --build`	Build images and start all containers
`docker compose down`	Stop and remove containers
`docker compose down -v`	Stop containers and remove volumes
`docker ps`	List running containers
`docker logs flask-app`	View Flask app logs
`docker images`	List all Docker images
`docker build -t flask-app .`	Build image only (no compose)
`docker system prune -af`	Remove unused Docker data

5. Application Endpoints

Route	Method	Description	Response
`/`	GET	Homepage	HTML page
`/health`	GET	Health check	{"status":"healthy"}
`/db`	GET	DB connection test	{"database":"connected"}

6. Verification — curl Output

All three endpoints tested successfully via curl in WSL Ubuntu terminal:

$ curl http://localhost:5000/

```
<h1>Alfido Tech DevOps Internship</h1><p>Flask app is running in Docker!</p>
```

$ curl http://localhost:5000/health

```
{"service": "flask-app", "status": "healthy"}
```

$ curl http://localhost:5000/db

```
{"database": "connected"}
```

7. Screenshot — Browser Verification

The following screenshot shows the Flask application running and accessible at http://localhost:5000 in the browser:

Figure 1: Flask app running at localhost:5000 in browser

8. Deliverables Checklist

Deliverable	Status
Dockerfile with multi-stage build	■ Complete
docker-compose.yml (app + PostgreSQL DB)	■ Complete
Flask app with 3 endpoints (/, /health, /db)	■ Complete
requirements.txt	■ Complete
.dockerignore	■ Complete
README.md	■ Complete
Container running and verified via curl	■ Complete
Browser screenshot at localhost:5000	■ Complete
GitHub repository push	■ Complete